Kodin Code Session Report - Parneet

Date: 20/10/2026, 9:18:45 pm

QUESTION

Section A: Theory (Questions 18) Upload Handwritten Notes as attachment in Kodin

1. Explain the architecture of the Java Virtual Machine (JVM). How does it enable platform independence? Describe the roles of the class loader, bytecode verifier, and execution engine using a real-world deployment analogy.

2. Explain the Java Collections Framework hierarchy. Using a Student Enrollment System as context, justify which of HashMap, HashSet, and TreeMap is best suited for fast lookup, unique records, and sorted output.

3. Explain encapsulation with a real-world example (such as a MobilePhone hiding its internals). Discuss JavaBeans and their connection to encapsulation, and explain why immutability is a special form of encapsulation, including its advantages in multi-threaded applications.

4. Differentiate between an abstract class and an interface with a real-world analogy. Explain the role of default and static methods in interfaces (Java 8+) and their impact on backward compatibility.

5. Why does Java not support multiple inheritance through classes? Explain the Diamond Problem with an example and show how interfaces resolve it.

6. Explain the Thread lifecycle (New, Runnable, Running, Blocked/Waiting, Terminated). Compare the two ways of creating threads: extending the Thread class vs implementing the Runnable interface.

7. Differentiate between sleep(), wait(), notify(), and notifyAll(). Explain how they are used for inter-thread communication.

8.Define race condition and deadlock with real-world examples. Explain the synchronized keyword (methods and blocks), the volatile keyword (when it is and is not sufficient), and how java.util.concurrent classes such as AtomicInteger and ConcurrentHashMap help.

Section B: Implementation in Kodin IDE (Questions 920)

9.A retail billing system stores product name, price, quantity, and total bill. Write a Java program using appropriate primitive and non-primitive data types.
Demonstrate type casting (implicit and explicit) within the billing calculation.

10.Differentiate between the == operator and the .equals() method. Using a Bank Account scenario, compare two account objects by reference and by content (account number), and explain the output.

11.Implement a BankAccount class that encapsulates balance, account number, and account holder name. The balance must be modified only through deposit() and withdraw() methods with business rule validations (e.g., no overdraft). Write a test class demonstrating the encapsulation.

12.Implement an immutable class ImmutableEmployee (name, empId, salary) whose fields cannot be modified once the object is created. Demonstrate that modification is not possible.

13.Design an abstract class PaymentProcessor with abstract methods initiatePayment(), validateTransaction(), and generateReceipt(). Implement four concrete subclasses: Credit Card, UPI, Net Banking, and Wallet.

14.A Smart Home system controls Lights, Fan, AirConditioner, and SecurityCamera. Create an interface SmartDevice with turnOn(), turnOff(), and getStatus(). Implement each device and control all of them using a List<SmartDevice> in a single loop.

15.Implement the hierarchy Employee Manager Director using multilevel inheritance. Each level overrides computeSalary() (team bonus for Manager, performance incentive for Director). Demonstrate polymorphism via a base class reference.

16.A banking application has Account as the base class with subclasses SavingsAccount, CurrentAccount, and LoanAccount. Use the instanceof operator and downcasting to perform type-specific operations. Demonstrate the risk of improper casting and how to avoid it.

17.Implement the Producer-Consumer problem using a shared buffer (an order queue in a restaurant kitchen). The chef (producer) and waiter (consumer) must coordinate using wait() and notify().

18.Simulate a deadlock in a Banking Transfer System where Thread A holds a lock on Account X and waits for Account Y, while Thread B holds Account Y and waits for Account X. Then resolve the deadlock using the lock ordering strategy.

19.Demonstrate a race condition in a Ticket Booking System where 10 threads simultaneously try to book the last available seat. Show the incorrect output, then fix it using the synchronized keyword and compare both outputs.

20.Using 10 threads, each doing 1000 increments on a shared Shopping Cart counter, compare a non-synchronized counter, a synchronized counter, and an AtomicInteger-based counter. Measure the execution time and correctness of each, and explain the results.

AIM

ALGORITHM

SOLUTIONS

File: code9.java

```
class Product
{
    String name;
    double price;
    int quantity;
    double total;
    Product(String name, double price, int quantity)
    {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
        this.total = price * quantity;
    }
}
public class code9
{
    public static void main(String[] args)
    {
        Product[] cart = {
            new Product("Notebook", 45.50, 4),
            new Product("Pen Box", 120.00, 2),
            new Product("Backpack", 899.99, 1)
        };
```

```
            double bill = 0;
            int totalItems = 0;
            System.out.println("Item          Price   Qty      Total");
            for (Product p : cart)
            {
                System.out.printf("%-10s %8.2f %5d %10.2f%n", p.name, p.price,
p.quantity, p.total);
                bill = bill + p.total;
                totalItems = totalItems + p.quantity;
            }
            System.out.printf("Bill before discount : %.2f%n", bill);
            int discountPercent = 10;
            double discount = bill * discountPercent / 100;
            double finalBill = bill - discount;
            System.out.printf("Discount (%d%%)       : %.2f%n", discountPercent,
discount);
            System.out.printf("Final bill           : %.2f%n", finalBill);
            long itemsAsLong = totalItems;
            float itemsAsFloat = totalItems;
            System.out.println("\nImplicit casting");
            System.out.println("int " + totalItems + " -> long " + itemsAsLong + " -
> float " + itemsAsFloat);
            int truncatedBill = (int) finalBill;
            int roundedBill = (int) Math.round(finalBill);
            float floatBill = (float) finalBill;
            System.out.println("\nExplicit casting");
            System.out.println("(int) finalBill             = " +
truncatedBill);
            System.out.println("(int) Math.round(finalBill) = " +
roundedBill);
            System.out.println("(float) finalBill           = " + floatBill);
        }
    }
```

File: code10.java

```
    class Account
    {
        private int accountNumber;
        private String holder;
        Account(int accountNumber, String holder)
        {
            this.accountNumber = accountNumber;
            this.holder = holder;
        }
        @Override
        public boolean equals(Object obj)
        {
            if (this == obj)
                return true;
            if (obj == null || getClass() != obj.getClass())
                return false;
            Account other = (Account) obj;
            return accountNumber == other.accountNumber;
        }
        @Override
        public int hashCode()
        {
            return Integer.hashCode(accountNumber);
        }
    }
    class PlainAccount
    {
        int accountNumber;
        PlainAccount(int accountNumber)
        {
```

```
            this.accountNumber = accountNumber;
        }
    }
    public class code10
    {
        public static void main(String[] args)
        {
            Account a1 = new Account(1001, "Aman");
            Account a2 = new Account(1001, "Aman");
            Account a3 = a1;
            Account a4 = new Account(1002, "Riya");
            System.out.println("a1 == a2      : " + (a1 == a2));
            System.out.println("a1.equals(a2) : " + a1.equals(a2));
            System.out.println("a1 == a3      : " + (a1 == a3));
            System.out.println("a1.equals(a4) : " + a1.equals(a4));
            PlainAccount p1 = new PlainAccount(2001);
            PlainAccount p2 = new PlainAccount(2001);
            System.out.println("p1.equals(p2) (no override) : " +
p1.equals(p2));
            String s1 = "java";
            String s2 = "java";
            String s3 = new String("java");
            System.out.println("s1 == s2      : " + (s1 == s2));
            System.out.println("s1 == s3      : " + (s1 == s3));
            System.out.println("s1.equals(s3) : " + s1.equals(s3));
        }
    }
```

File: code11.java

```
    class BankAccount
    {
        private double balance;
        private final int accountNumber;
        private String holderName;
        BankAccount(int accountNumber, String holderName, double openingBalance)
        {
            this.accountNumber = accountNumber;
            this.holderName = holderName;
            this.balance = openingBalance >= 0 ? openingBalance : 0;
        }
        public void deposit(double amount)
        {
            if (amount <= 0)
            {
                System.out.println("Deposit rejected: amount must be
positive");
                return;
            }
            balance = balance + amount;
            System.out.println("Deposited " + amount + " | Balance: " + balance);
        }
        public void withdraw(double amount)
        {
            if (amount <= 0)
            {
                System.out.println("Withdraw rejected: amount must be
positive");
            }
            else if (amount > balance)
            {
                System.out.println("Withdraw rejected: insufficient balance (no
overdraft)");
            }
            else
            {
```

```
                balance = balance - amount;
                System.out.println("Withdrew " + amount + " | Balance: " +
balance);
            }
        }
        public double getBalance()
        {
            return balance;
        }
        public int getAccountNumber()
        {
            return accountNumber;
        }
        public String getHolderName()
        {
            return holderName;
        }
    }
    public class code11
    {
        public static void main(String[] args)
        {
            BankAccount acc = new BankAccount(5001, "Parneet Kaur", 1000);
            System.out.println("Holder: " + acc.getHolderName() + ", Account: " +
acc.getAccountNumber());
            System.out.println("Opening balance: " + acc.getBalance());
            acc.deposit(500);
            acc.withdraw(300);
            acc.withdraw(5000);
            acc.deposit(-50);
            acc.withdraw(0);
            System.out.println("Final balance: " + acc.getBalance());
        }
    }
```

File: code12.java

```
    final class ImmutableEmployee
    {
        private final String name;
        private final int empId;
        private final double salary;
        ImmutableEmployee(String name, int empId, double salary)
        {
            this.name = name;
            this.empId = empId;
            this.salary = salary;
        }
        public String getName()
        {
            return name;
        }
        public int getEmpId()
        {
            return empId;
        }
        public double getSalary()
        {
            return salary;
        }
        public ImmutableEmployee withSalary(double newSalary)
        {
            return new ImmutableEmployee(name, empId, newSalary);
        }
        public String toString()
        {
```

```
            return empId + " | " + name + " | " + salary;
        }
    }
    public class code12
    {
        public static void main(String[] args)
        {
            ImmutableEmployee e1 = new ImmutableEmployee("Aman Verma", 101,
40000);
            System.out.println("e1 before : " + e1);
            ImmutableEmployee e2 = e1.withSalary(55000);
            System.out.println("e1 after  : " + e1);
            System.out.println("e2 (new)  : " + e2 );
            System.out.println("Same object? " + (e1 == e2));
        }
    }
```

File: code13.java

```
    abstract class PaymentProcessor
    {
        protected double amount;
        protected String transactionId;
        PaymentProcessor(String transactionId, double amount)
        {
            this.transactionId = transactionId;
            this.amount = amount;
        }
        abstract void initiatePayment();
        abstract boolean validateTransaction();
        abstract void generateReceipt();
        final void process()
        {
            System.out.println("---- Transaction " + transactionId + "
----");
            initiatePayment();
            if (validateTransaction())
            {
                generateReceipt();
            }
            else
            {
                System.out.println("Payment FAILED for " + transactionId + ". No
receipt generated.");
            }        System.out.println();
        }
    }
    class CreditCardPayment extends PaymentProcessor
    {
        private String cardNumber;
        private int cvv;
        CreditCardPayment(String id, double amount, String cardNumber, int cvv)
        {
            super(id, amount);
            this.cardNumber = cardNumber;
            this.cvv = cvv;
        }
        void initiatePayment()
        {
            System.out.println("Credit Card: charging " + amount + " to card ending
" + cardNumber.substring(cardNumber.length() - 4));
        }
        boolean validateTransaction()
        {
            return cardNumber.length() == 16 && cvv >= 100 && cvv <= 999 && amount
<= 50000;
```

```
        }
        void generateReceipt()
        {
            System.out.println("Receipt [Credit Card] " + transactionId + " : " +
amount + " paid");
        }
    }
    class UPIPayment extends PaymentProcessor
    {
        private String upiId;
        UPIPayment(String id, double amount, String upiId)
        {
            super(id, amount);
            this.upiId = upiId;
        }
        void initiatePayment()
        {
            System.out.println("UPI: collect request of " + amount + " sent to " +
upiId);
        }
        boolean validateTransaction()
        {
            return upiId.contains("@") && amount <= 100000;
        }
        void generateReceipt()
        {
            System.out.println("Receipt [UPI] " + transactionId + " : " + amount + "
paid by " + upiId);
        }
    }
    class NetBankingPayment extends PaymentProcessor
    {
        private String bankName;
        private String userId;
        NetBankingPayment(String id, double amount, String bankName, String
userId)
        {
            super(id, amount);
            this.bankName = bankName;
            this.userId = userId;
        }
        void initiatePayment()
        {
            System.out.println("Net Banking: redirecting " + userId + " to " +
bankName + " portal");
        }
        boolean validateTransaction()
        {
            return !userId.isEmpty() && amount > 0;
        }
        void generateReceipt()
        {
            System.out.println("Receipt [Net Banking] " + transactionId + " : " +
amount + " paid via " + bankName);
        }
    }
    class WalletPayment extends PaymentProcessor
    {
        private double walletBalance;
        WalletPayment(String id, double amount, double walletBalance)
        {
            super(id, amount);
            this.walletBalance = walletBalance;
        }
        void initiatePayment()
```

```
        {
            System.out.println("Wallet: debiting " + amount + " from balance " +
walletBalance);
        }
        boolean validateTransaction()
        {
            return walletBalance >= amount;
        }
        void generateReceipt()
        {
            System.out.println("Receipt [Wallet] " + transactionId + " : " + amount
+ " paid, balance left " + (walletBalance - amount));
        }
    }
    public class code13
    {
        public static void main(String[] args)
        {
            PaymentProcessor[] payments = {
                new CreditCardPayment("TXN101", 2500, "4111111111111111",
123),
                new UPIPayment("TXN102", 799, "parneet@upi"),
                new NetBankingPayment("TXN103", 12000, "SBI",
"parneet01"),
                new WalletPayment("TXN104", 900, 450)
            };
            for (PaymentProcessor p : payments)
            {
                p.process();
            }
        }
    }
```

File: code14.java

```
    import java.util.ArrayList;
    import java.util.List;
    interface SmartDevice
    {
        void turnOn();
        void turnOff();
        String getStatus();
    }
    class Light implements SmartDevice
    {
        private boolean on;
        public void turnOn()
        {
            on = true;
            System.out.println("Light switched ON");
        }
        public void turnOff()
        {
            on = false;
            System.out.println("Light switched OFF");
        }
        public String getStatus()
        {
            return "Light is " + (on ? "ON" : "OFF");
        }
    }
    class Fan implements SmartDevice
    {
        private boolean on;
        private int speed;
        public void turnOn()
```

```
    {
        on = true;
        speed = 3;
        System.out.println("Fan started at speed 3");
    }
    public void turnOff()
    {
        on = false;
        speed = 0;
        System.out.println("Fan stopped");
    }
    public String getStatus()
    {
        return "Fan is " + (on ? "ON at speed " + speed : "OFF");
    }
}
class AirConditioner implements SmartDevice
{
    private boolean on;
    private int temperature = 24;
    public void turnOn()
    {
        on = true;
        System.out.println("AC cooling to " + temperature + " C");
    }
    public void turnOff()
    {
        on = false;
        System.out.println("AC switched OFF");
    }
    public String getStatus()
    {
        return "AC is " + (on ? "ON set to " + temperature + " C" : "OFF");
    }
}
class SecurityCamera implements SmartDevice
{
    private boolean recording;
    public void turnOn()
    {
        recording = true;
        System.out.println("Camera recording started");
    }
    public void turnOff()
    {
        recording = false;
        System.out.println("Camera recording stopped");
    }
    public String getStatus()
    {
        return "Camera is " + (recording ? "RECORDING" : "IDLE");
    }
}
public class code14
{
    public static void main(String[] args)
    {
        List<SmartDevice> devices = new ArrayList<>();
        devices.add(new Light());
        devices.add(new Fan());
        devices.add(new AirConditioner());
        devices.add(new SecurityCamera());
        System.out.println("== Turning everything on ==");
        for (SmartDevice d : devices)
        {
```

```
                d.turnOn();
            }
            System.out.println("\n== Status ==");
            for (SmartDevice d : devices)
            {
                System.out.println(d.getStatus());
            }
            System.out.println("\n== Turning everything off ==");
            for (SmartDevice d : devices)
            {
                d.turnOff();
            }
            System.out.println("\n== Status ==");
            for (SmartDevice d : devices)
            {
                System.out.println(d.getStatus());
            }
        }
    }
```

File: code15.java

```
    class Employee
    {
        protected String name;
        protected double basicSalary;
        Employee(String name, double basicSalary)
        {
            this.name = name;
            this.basicSalary = basicSalary;
        }
        double computeSalary()
        {
            return basicSalary;
        }
        void show()
        {
            System.out.println(getClass().getSimpleName() + " " + name + " earns " +
computeSalary());
        }
    }
    class Manager extends Employee
    {
        protected int teamSize;
        Manager(String name, double basicSalary, int teamSize)
        {
            super(name, basicSalary);
            this.teamSize = teamSize;
        }
        @Override
        double computeSalary()
        {
            double teamBonus = teamSize * 2000;
            return super.computeSalary() + teamBonus;
        }
    }
    class Director extends Manager
    {
        private int performanceRating;
        Director(String name, double basicSalary, int teamSize, int
performanceRating)
        {
            super(name, basicSalary, teamSize);
            this.performanceRating = performanceRating;
        }
        @Override
```

```
    double computeSalary()
    {
        double incentive = performanceRating * 10000;
        return super.computeSalary() + incentive;
    }
}
public class code15
{
    public static void main(String[] args)
    {
        Employee e = new Employee("Aman", 30000);
        Employee m = new Manager("Riya", 50000, 5);
        Employee d = new Director("Kabir", 90000, 12, 4);
        Employee[] staff = {e, m, d};
        for (Employee emp : staff)
        {
            emp.show();
        }
    }
}
```

File: code16.java

```
class Account
{
    protected int accNo;
    protected double balance;
    Account(int accNo, double balance)
    {
        this.accNo = accNo;
        this.balance = balance;
    }
    void showBalance()
    {
        System.out.println("Account " + accNo + " balance: " + balance);
    }
}
class SavingsAccount extends Account
{
    private double rate = 0.04;
    SavingsAccount(int accNo, double balance)
    {
        super(accNo, balance);
    }
    void addInterest()
    {
        balance = balance + balance * rate;
        System.out.println("  Interest added, new balance: " + balance);
    }
}
class CurrentAccount extends Account
{
    private double overdraftLimit = 10000;
    CurrentAccount(int accNo, double balance)
    {
        super(accNo, balance);
    }
    void showOverdraft()
    {
        System.out.println("  Overdraft limit: " + overdraftLimit);
    }
}
class LoanAccount extends Account
{
    private double emi = 5000;
    LoanAccount(int accNo, double balance)
```

```
        {
            super(accNo, balance);
        }
        void payEmi()
        {
            balance = balance - emi;
            System.out.println("  EMI paid, outstanding loan: " + balance);
        }
    }
    public class code16
    {
        public static void main(String[] args)
        {
            Account[] accounts = {
                new SavingsAccount(1, 20000),
                new CurrentAccount(2, 15000),
                new LoanAccount(3, 100000)
            };
            System.out.println("== Safe downcasting with instanceof ==");
            for (Account a : accounts)
            {
                a.showBalance();
                if (a instanceof SavingsAccount)
                {
                    SavingsAccount s = (SavingsAccount) a;
                    s.addInterest();
                }
                else if (a instanceof CurrentAccount)
                {
                    CurrentAccount c = (CurrentAccount) a;
                    c.showOverdraft();
                }
                else if (a instanceof LoanAccount)
                {
                    LoanAccount l = (LoanAccount) a;
                    l.payEmi();
                }
            }
            System.out.println("\n== Improper casting ==");
            Account risky = accounts[0];
            try
            {
                LoanAccount wrong = (LoanAccount) risky;
                wrong.payEmi();
            }
            catch (ClassCastException e)
            {
                System.out.println("ClassCastException: a SavingsAccount object
cannot be treated as a LoanAccount");
            }
            System.out.println("\n== Avoiding it ==");
            if (risky instanceof LoanAccount)
            {
                ((LoanAccount) risky).payEmi();
            }
            else
            {
                System.out.println("Cast skipped: object is not a
LoanAccount");
            }
        }
    }
```

File: code17.java

```
    import java.util.LinkedList;
```

```
    class OrderQueue
    {
        private final LinkedList<String> orders = new LinkedList<>();
        private final int capacity;    OrderQueue(int capacity)
        {
            this.capacity = capacity;
        }
        synchronized void placeOrder(String dish) throws InterruptedException
        {
            while (orders.size() == capacity)
            {
                System.out.println("Counter full, chef waits...");
                wait();
            }
            orders.addLast(dish);
            System.out.println("Chef prepared  : " + dish + "   [queue size " +
orders.size() + "]");
            notifyAll();
        }
        synchronized String serveOrder() throws InterruptedException
        {
            while (orders.isEmpty())
            {
                System.out.println("Counter empty, waiter waits...");
                wait();
            }
            String dish = orders.removeFirst();
            System.out.println("Waiter served  : " + dish + "   [queue size " +
orders.size() + "]");
            notifyAll();
            return dish;
        }
    }
    class Chef extends Thread
    {
        private OrderQueue queue;
        Chef(OrderQueue queue)
        {
            this.queue = queue;
        }
        public void run()
        {
            String[] menu = {"Pizza", "Pasta", "Burger", "Noodles", "Paneer Tikka",
"Dosa"};
            try
            {
                for (String dish : menu)
                {
                    queue.placeOrder(dish);
                    Thread.sleep(200);
                }
            }
            catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    }
    class Waiter extends Thread
    {
        private OrderQueue queue;
        Waiter(OrderQueue queue)
        {
            this.queue = queue;
        }
```

```
        public void run()
        {
            try
            {
                for (int i = 0; i < 6; i++)
                {
                    queue.serveOrder();
                    Thread.sleep(600);
                }
            }
            catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    }
    public class code17
    {
        public static void main(String[] args) throws InterruptedException
        {
            OrderQueue queue = new OrderQueue(2);
            Chef chef = new Chef(queue);
            Waiter waiter = new Waiter(queue);
            chef.start();
            waiter.start();
            chef.join();
            waiter.join();
            System.out.println("All orders served.");
        }
    }
```

File: code18.java

```
    import java.lang.management.ManagementFactory;
    import java.lang.management.ThreadMXBean;
    class Account
    {
        final int id;
        double balance;
        Account(int id, double balance)
        {
            this.id = id;
            this.balance = balance;
        }
    }
    public class code18
    {
        static void pause(int ms)
        {
            try
            {
                Thread.sleep(ms);
            }
            catch (InterruptedException e)
            {

            }
        }
        static void unsafeTransfer(Account from, Account to, double amount)
        {
            synchronized (from)
            {
                System.out.println(Thread.currentThread().getName() + " locked
Account " + from.id + ", waiting for Account " + to.id);
                pause(200);
                synchronized (to)
```

```
                {
                    from.balance -= amount;
                    to.balance += amount;
                    System.out.println(Thread.currentThread().getName() + "
transferred " + amount);
                }
            }
        }
        static void orderedTransfer(Account from, Account to, double amount)
        {
            Account first = from.id < to.id ? from : to;
            Account second = from.id < to.id ? to : from;
            synchronized (first)
            {
                System.out.println(Thread.currentThread().getName() + " locked
Account " + first.id + ", next Account " + second.id);
                pause(200);
                synchronized (second)
                {
                    from.balance -= amount;
                    to.balance += amount;
                    System.out.println(Thread.currentThread().getName() + "
transferred " + amount);
                }
            }
        }
        public static void main(String[] args) throws InterruptedException
        {
            System.out.println("=== Part 1: deadlock ===");
            Account x = new Account(1, 5000);
            Account y = new Account(2, 5000);
            Thread a = new Thread(() -> unsafeTransfer(x, y, 100), "Thread-
A");
            Thread b = new Thread(() -> unsafeTransfer(y, x, 200), "Thread-
B");
            a.setDaemon(true);
            b.setDaemon(true);
            a.start();
            b.start();
            a.join(2000);
            b.join(2000);
            System.out.println("Thread-A state: " + a.getState());
            System.out.println("Thread-B state: " + b.getState());
            ThreadMXBean bean = ManagementFactory.getThreadMXBean();
            long[] stuck = bean.findDeadlockedThreads();
            System.out.println("Deadlocked threads found: " + (stuck == null ? 0 :
stuck.length));
            System.out.println("\n=== Part 2: fixed with lock ordering ===");
            Account p = new Account(1, 5000);
            Account q = new Account(2, 5000);
            Thread c = new Thread(() -> orderedTransfer(p, q, 100), "Thread-
C");
            Thread d = new Thread(() -> orderedTransfer(q, p, 200), "Thread-
D");
            c.start();
            d.start();
            c.join();
            d.join();
            System.out.println("Account 1 balance: " + p.balance);
            System.out.println("Account 2 balance: " + q.balance);
            System.out.println("Both threads finished, no deadlock.");
        }
    }
```

File: code19.java

```
import java.util.concurrent.atomic.AtomicInteger;
class TicketCounter
{
    private int seats = 1;
    private final AtomicInteger bookings = new AtomicInteger();
    void bookUnsafe(String user)
    {
        if (seats > 0)
        {
            pause();
            seats--;
            bookings.incrementAndGet();
            System.out.println(user + " booked the seat");
        }
        else
        {
            System.out.println(user + " found no seat");
        }
    }
    synchronized void bookSafe(String user)
    {
        if (seats > 0)
        {
            pause();
            seats--;
            bookings.incrementAndGet();
            System.out.println(user + " booked the seat");
        }
        else
        {
            System.out.println(user + " found no seat");
        }
    }
    int getSeats()
    {
        return seats;
    }
    int getBookings()
    {
        return bookings.get();
    }
    private void pause()
    {
        try
        {
            Thread.sleep(50);
        }
        catch (InterruptedException e) { }
    }
}
class Customer extends Thread
{
    private TicketCounter counter;
    private boolean safe;
    Customer(String name, TicketCounter counter, boolean safe)
    {
        super(name);
        this.counter = counter;
        this.safe = safe;
    }
    public void run()
    {
        if (safe)
            counter.bookSafe(getName());
        else
```

```
                counter.bookUnsafe(getName());
            }
        }
        public class code19
        {
            static void simulate(boolean safe) throws InterruptedException
            {
                TicketCounter counter = new TicketCounter();
                Customer[] users = new Customer[10];
                for (int i = 0; i < 10; i++)
                {
                    users[i] = new Customer("User-" + (i + 1), counter, safe);
                }
                for (Customer u : users)
                    u.start();
                for (Customer u : users)
                    u.join();
                System.out.println("Seats available at the start : 1");
                System.out.println("Successful bookings          : " +
counter.getBookings());
                System.out.println("Seats left                   : " +
counter.getSeats());
            }
            public static void main(String[] args) throws InterruptedException
            {
                System.out.println("=== Without synchronization ===");
                simulate(false);
                System.out.println("\n=== With synchronized ===");
                simulate(true);
            }
        }
```

File: code20.java

```
        import java.util.concurrent.CountDownLatch;
        import java.util.concurrent.atomic.AtomicInteger;
        interface Counter
        {
            void increment();
            int get();
        }
        class UnsafeCounter implements Counter
        {
            private int count = 0;
            public void increment()
            {
                count++;
            }
            public int get()
            {
                return count;
            }
        }
        class SyncCounter implements Counter
        {
            private int count = 0;
            public synchronized void increment()
            {
                count++;
            }
            public synchronized int get()
            {
                return count;
            }
        }
        class AtomicCounter implements Counter
```

```
    {
        private AtomicInteger count = new AtomicInteger(0);
        public void increment()
        {
            count.incrementAndGet();
        }
        public int get()
        {
            return count.get();
        }
    }
    class Shopper implements Runnable
    {
        private Counter counter;
        private CountDownLatch startSignal;
        Shopper(Counter counter, CountDownLatch startSignal)
        {
            this.counter = counter;
            this.startSignal = startSignal;
        }
        public void run()
        {
            try
            {
                startSignal.await();
            }
            catch (InterruptedException e) { }
            for (int i = 0; i < 1000; i++)
            {
                counter.increment();
            }
        }
    }
    public class code20
    {
        static void test(String label, Counter counter) throws
InterruptedException
        {
            CountDownLatch startSignal = new CountDownLatch(1);
            Thread[] threads = new Thread[10];
            for (int i = 0; i < 10; i++)
            {
                threads[i] = new Thread(new Shopper(counter, startSignal));
            }
            for (Thread t : threads) t.start();
            long start = System.nanoTime();
            startSignal.countDown();
            for (Thread t : threads) t.join();
            long end = System.nanoTime();
            int result = counter.get();
            System.out.printf("%-14s final = %5d  expected = 10000  correct = %-5b
time = %.3f ms%n",                label, result, result == 10000, (end - start) /
1_000_000.0);
        }
        public static void main(String[] args) throws InterruptedException
        {
            test("Unsynchronized", new UnsafeCounter());
            test("Synchronized", new SyncCounter());
            test("AtomicInteger", new AtomicCounter());
        }
    }
```

PROGRAM OUTPUT

```
    ========== code9.java ==========
```

```
Item          Price   Qty      Total
Notebook      45.50     4     182.00
Pen Box      120.00     2     240.00
Backpack     899.99     1     899.99
Bill before discount : 1321.99
Discount (10%)       : 132.20
Final bill           : 1189.79Implicit casting
int 7 -> long 7 -> float 7.0Explicit casting
(int) finalBill             = 1189
(int) Math.round(finalBill) = 1190
(float) finalBill           = 1189.791

========== code10.java ==========
a1 == a2      : false
a1.equals(a2) : true
a1 == a3      : true
a1.equals(a4) : false
p1.equals(p2) (no override) : false
s1 == s2      : true
s1 == s3      : false
s1.equals(s3) : true

========== code11.java ==========
Holder: Parneet Kaur, Account: 5001
Opening balance: 1000.0
Deposited 500.0 | Balance: 1500.0
Withdrew 300.0 | Balance: 1200.0
Withdraw rejected: insufficient balance (no overdraft)
Deposit rejected: amount must be positive
Withdraw rejected: amount must be positive
Final balance: 1200.0

========== code12.java ==========
e1 before : 101 | Aman Verma | 40000.0
e1 after  : 101 | Aman Verma | 40000.0
e2 (new)  : 101 | Aman Verma | 55000.0
Same object? false

========== code13.java ==========
---- Transaction TXN101 ----
Credit Card: charging 2500.0 to card ending 1111
Receipt [Credit Card] TXN101 : 2500.0 paid---- Transaction TXN102 ----
UPI: collect request of 799.0 sent to parneet@upi
Receipt [UPI] TXN102 : 799.0 paid by parneet@upi---- Transaction TXN103 ----
Net Banking: redirecting parneet01 to SBI portal
Receipt [Net Banking] TXN103 : 12000.0 paid via SBI---- Transaction TXN104 ----
Wallet: debiting 900.0 from balance 450.0
Payment FAILED for TXN104. No receipt generated.

========== code14.java ==========
== Turning everything on ==
Light switched ON
Fan started at speed 3
AC cooling to 24 C
Camera recording started== Status ==
Light is ON
Fan is ON at speed 3
AC is ON set to 24 C
Camera is RECORDING== Turning everything off ==
Light switched OFF
Fan stopped
AC switched OFF
Camera recording stopped== Status ==
Light is OFF
Fan is OFF
```

```
    AC is OFF
    Camera is IDLE

    ========== code15.java ==========
    Employee Aman earns 30000.0
    Manager Riya earns 60000.0
    Director Kabir earns 154000.0

    ========== code16.java ==========
    == Safe downcasting with instanceof ==
    Account 1 balance: 20000.0
      Interest added, new balance: 20800.0
    Account 2 balance: 15000.0
      Overdraft limit: 10000.0
    Account 3 balance: 100000.0
      EMI paid, outstanding loan: 95000.0== Improper casting ==
    ClassCastException: a SavingsAccount object cannot be treated as a LoanAccount==
Avoiding it ==
    Cast skipped: object is not a LoanAccount

    ========== code17.java ==========
    Chef prepared  : Pizza   [queue size 1]
    Waiter served  : Pizza   [queue size 0]
    Chef prepared  : Pasta   [queue size 1]
    Chef prepared  : Burger   [queue size 2]
    Counter full, chef waits...
    Waiter served  : Pasta   [queue size 1]
    Chef prepared  : Noodles   [queue size 2]
    Counter full, chef waits...
    Waiter served  : Burger   [queue size 1]
    Chef prepared  : Paneer Tikka   [queue size 2]
    Counter full, chef waits...
    Waiter served  : Noodles   [queue size 1]
    Chef prepared  : Dosa   [queue size 2]
    Waiter served  : Paneer Tikka   [queue size 1]
    Waiter served  : Dosa   [queue size 0]
    All orders served.

    ========== code18.java ==========
    === Part 1: deadlock ===
    Thread-B locked Account 2, waiting for Account 1
    Thread-A locked Account 1, waiting for Account 2
    Thread-A state: BLOCKED
    Thread-B state: BLOCKED
    Deadlocked threads found: 2

    === Part 2: fixed with lock ordering ===
    Thread-C locked Account 1, next Account 2
    Thread-C transferred 100.0
    Thread-D locked Account 1, next Account 2
    Thread-D transferred 200.0
    Account 1 balance: 5100.0
    Account 2 balance: 4900.0
    Both threads finished, no deadlock.

    ========== code19.java ==========
    === Without synchronization ===
    User-9 booked the seat
    User-4 booked the seat
    User-7 booked the seat
    User-3 booked the seat
    User-10 booked the seat
    User-1 booked the seat
    User-2 booked the seat
    User-8 booked the seat
```

```
User-5 booked the seat
User-6 booked the seat
Seats available at the start : 1
Successful bookings          : 10
Seats left                   : -9=== With synchronized ===
User-1 booked the seat
User-10 found no seat
User-9 found no seat
User-8 found no seat
User-7 found no seat
User-6 found no seat
User-5 found no seat
User-4 found no seat
User-3 found no seat
User-2 found no seat
Seats available at the start : 1
Successful bookings          : 1
Seats left                   : 0

========== code20.java ==========
Unsynchronized final =  8663  expected = 10000  correct = false  time = 3.811 ms
Synchronized   final = 10000  expected = 10000  correct = true   time = 3.066 ms
AtomicInteger  final = 10000  expected = 10000  correct = true   time = 2.395 ms
```

CONCLUSION

Downcasting gives access to subclass methods that the base reference cannot see, and instanceof tells which cast is safe. A wrong cast compiles but throws ClassCastException at run time, because the compiler only checks that the types are related. Testing with instanceof before casting avoids the crash.

External Activity / Applications Found During Session

1:53:41 pm Clipboard action blocked

2:22:18 pm Visited outside Kodin

8:05:32 pm msedge